

Paper 5 - Oloid Path Carrier

CQE-paper-05

CQECMPLX Formal Paper Series

Review-facing PDF built 2026-06-12

Construction Basis

Use curved/rolling carriers to show that transport need not be straight-line to preserve continuity.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- top-level review source: Papers\Source\CQE-paper-05.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-05\paper_kernel_manifest.json
- body: production\papers\CQE-paper-05\PAPER-BODY.md
- source: production\papers\CQE-paper-05\SOURCE.md
- formal: production\papers\CQE-paper-05\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-05\02-CQE-tool\TOOL.md
- tool_runner: production\papers\CQE-paper-05\02-CQE-tool\run.py
- workbook: production\papers\CQE-paper-05\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-05.md

Paper 5 - Oloid Path Carrier

Abstract

Paper 5 proves the first path-carrier theorem in the CQECMPLX sequence. Paper 4 converts a failed join into a typed boundary constraint. Paper 5 proves that such a constraint can be carried along a legal rolling path without requiring straight-line transport and without rewriting the path rule.

The verified claim is structural:

```
legal rolling steps preserve path continuity
head/tail dyads remain binary
Paper 4 constraints can be attached as payloads
payloads do not alter the carrier rule
```

This paper does not prove that the Oloid carrier predicts Rule 30 future bits. It proves the carrier property needed before such a prediction claim could be tested.

Claims

Claim 5.1. For every finite binary input stream, the rolling carrier produces a trace whose adjacent states are legal rolling steps.

Claim 5.2. The head/tail dyad derived from each rolling state is binary.

Claim 5.3. A Paper 4 repair constraint can be attached to a trace state as a payload without changing the rolling rule.

Claim 5.4. Invalid bits and discontinuous jumps are rejected.

Definitions

A **rolling carrier state** is:

```
q = (sheet, phase, parity)
sheet in {0,1}
phase in {0,1,2,3}
parity in {0,1}
```

For a bit b in $\{0,1\}$, define:

```
roll(q,b) = ((sheet+1) mod 2, (phase+1) mod 4, parity xor b)
```

The **head/tail dyad** is:

```
head = sheet
tail = (phase mod 2) xor sheet xor parity
```

A **continuous carrier trace** is a finite state list in which every adjacent pair is related by one legal `roll` step for the corresponding input bit.

Theorem 5.1 - Rolling Path Carrier

For every finite binary input stream, the rolling carrier produces a continuous trace of legal adjacent states. The trace preserves input order, maintains a binary head/tail dyad at every state, and can carry Paper 4 constraints as receipt payloads without treating the path as a straight-line jump.

Proof

The carrier state space is finite:

```
{0,1} × {0,1,2,3} × {0,1}
```

For every valid bit `b`, the rule is total on that state space. It flips `sheet`, advances `phase` by one modulo four, and updates `parity` by XOR with `b`. Therefore every valid input bit gives exactly one legal successor.

A trace generated by repeated applications of `roll` is continuous by construction: each adjacent pair is one legal step apart. The head/tail dyad is a deterministic binary function of the current state, so it is defined at every trace position and stays in $\{0,1\}^2$.

A Paper 4 repair constraint can be attached to a state as payload because the state and input index are replayable. The payload does not enter the `roll` definition, so it cannot alter the legal successor relation. Thus payload transport preserves the path rule.

Worked Trace

For input bits:

```
[1,0,1,1,0,0,1,0]
```

from initial state `(0,0,0)`, the verifier records:

```
(0,0,0)
(1,1,1)
(0,2,1)
(1,3,0)
(0,0,1)
(1,1,1)
(0,2,1)
(1,3,0)
(0,0,0)
```

A Paper 4 constraint may be attached at `(1,3,0)` with head/tail `(1,0)` without changing the next rolling state.

Receipt

The primary executable receipt is:

```
production/formal-papers/CQE-paper-05/verify_oloid_path_carrier.py
production/formal-papers/CQE-paper-05/oloid_path_carrier_receipt.json
```

The receipt status is `pass`. It verifies:

```
trace_length_is_input_length_plus_one = true
adjacent_pairs_are_legal_rolls         = true
head_tail_dyads_are_binary              = true
payload_does_not_alter_path_rule        = true
invalid_bit_rejected                    = true
discontinuous_jump_rejected              = true
prediction_claim_left_out_of_scope       = true
```

Scope Boundary

This paper does not prove:

```
that the Oloid carrier predicts Rule 30 future bits
that physical Oloid geometry has been fully formalized here
that every curved path is a valid carrier
```

Those claims require separate receipts.

Falsifiers

The paper fails if any of the following occur:

```
an adjacent trace pair is not generated by one legal roll
a nonbinary input is accepted
a head/tail value leaves {0,1}
a payload changes the rolling rule
a discontinuous jump is accepted as legal
prediction is counted as closed by this receipt alone
```

Role in the Suite

Paper 4 produces the object Paper 5 can carry:

```
repair_boundary(s) = constraint row r
attach_payload(q_t, r) = carried receipt at path state q_t
payload_alters_path_rule = false
```

Paper 5 then exports a carried path state to Paper 6, where the dependency between repair, path, and receipt becomes a typed causal edge.

Open Obligations

- Expose `verify\_oloid\_path\_carrier` through the installable kernel/API interface.
- Bind Paper 4 repair payloads to the shared route ledger.
- Keep finite rolling-state claims separate from physical Oloid geometry claims until the latter have their own verifiers.
- Keep Rule 30 prediction open until a verifier closes it.

Conclusion

Paper 5 proves that transport can be continuous without being straight-line. A finite rolling carrier advances by legal steps, emits binary dyads, and carries repair constraints without changing its rule. That is the path substrate the later causal and accumulated-center papers need.